Auditoria de Normalização — Diário de Bordo Coletivo

Disciplina: Modelagem de Dados · **Projeto:** ERP para Micro e Pequenas Empresas · **Aula 08 — Normalização de Dados (1FN, 2FN e 3FN)**

Produto obrigatório: modelo lógico revisado e normalizado até a 3FN, com o dicionário de dados atualizado e as justificativas técnicas.

1. Identificação da equipe

Data	Aula de 01/10/2026 — preenchido em 26/10/2026, após a aula
Grupo	7
Empresa / projeto ERP	Petly Club — ERP para pet shop com loja física e loja virtual
Responsável pelo preenchimento	Gabriela Colombo

> **Observação:** este diário não foi entregue no dia da aula. Ele foi preenchido em 26/10/2026, com base no modelo revisado depois da devolutiva da primeira entrega.

2. Missão da equipe

Auditar a estrutura do banco do ERP antes da implementação: identificar problemas de normalização, propor correções e garantir que o modelo lógico esteja na 3FN, analisando no mínimo quatro tabelas, incluindo uma associativa.

3. Seleção das tabelas

Tabela	Classificação	
Tabela 1	PEDIDO(id_pedido, data_pedido, canal_venda, tipo_recebimento, status, desconto, id_cliente, id_funcionario)	Entidade forte com FKs para CLIENTE e FUNCIONÁRIO
Tabela 2	ITEM_PEDIDO(id_pedido, id_produto, quantidade, preco_unitario)	Tabela associativa (resolve o N:N Pedido × Produto); PK composta
Tabela 3	PRODUTO(id_produto, nome, descricao, marca, codigo_barras, preco, estoque_minimo, controla_validade, id_categoria)	Entidade forte com FK para CATEGORIA
Tabela 4	ENTREGA(id_entrega, logradouro, numero, complemento, bairro, cidade, uf, cep, data_prevista, data_entrega, status, id_pedido)	Entidade forte; 1:1 opcional com PEDIDO (id_pedido UNIQUE)

4. Auditoria de normalização

☑ Os atributos apresentam valores atômicos, sem listas ou grupos repetitivos (1FN).
☑ Todos os atributos não-chave dependem da chave completa, sem dependências parciais (2FN).
☑ Não existem dependências transitivas indevidas entre atributos não-chave (3FN).
☑ As dependências funcionais foram identificadas corretamente.

☑ Não existem redundâncias desnecessárias.
☑ Foram analisados os riscos de anomalias de inserção, atualização e exclusão.
☑ As chaves primárias e estrangeiras estão corretamente definidas.
☑ A decomposição preserva os relacionamentos e as regras de negócio.

8 de 8 critérios verificados (depois das correções registradas abaixo).

Dependências funcionais identificadas:

- PEDIDO: id_pedido → data_pedido, canal_venda, tipo_recebimento, status, desconto, id_cliente, id_funcionario
- ITEM_PEDIDO: (id_pedido, id_produto) → quantidade, preco_unitario
- PRODUTO: id_produto → nome, descricao, marca, codigo_barras, preco, estoque_minimo, controla_validade, id_categoria; codigo_barras → id_produto (chave candidata)
- ENTREGA: id_entrega → todos os demais atributos; id_pedido → id_entrega (chave candidata, UNIQUE)

5. Registro das inconsistências encontradas

Registro 1 — ENTREGA (endereço)
Foi verificado se o endereço de entrega estava guardado como um único texto (“Rua X, 100, ap 12, Centro, São Paulo – SP”), o que não seria atômico e impediria filtrar entregas por bairro ou cidade e validar o CEP. O grupo já tinha modelado o endereço como atributo composto, decomposto em logradouro, numero, complemento, bairro, cidade, uf e cep. Também não há grupos repetitivos: cada pedido tem no máximo uma entrega. Resultado: a tabela já atende à 1FN; nenhuma correção necessária.
Forma normal analisada: 1FN

Registro 2 — ITEM_PEDIDO (preço e nome do produto)
Foi analisado se algum atributo do item depende só de id_produto, parte da chave. O nome do produto dependeria (id_produto → nome) e por isso não foi colocado no item: ele fica em PRODUTO. Já o preco_unitario parece depender só do produto, mas é o preço praticado naquela venda (RN13). Ele depende do pedido e do produto juntos e pode ser diferente do preço atual. Resultado: não há dependência parcial; preco_unitario fica no item.
Forma normal analisada: 2FN

Registro 3 — ITEM_PEDIDO e PEDIDO (subtotal e valor total)
No rascunho da aula 5, o item tinha subtotal, e o pedido tinha valor_total. O subtotal depende de quantidade e preco_unitario (atributos não-chave), e o valor_total depende dos itens e do desconto. Guardar esses valores cria dependência transitiva e risco de o total não bater com os itens. Correção: os dois são atributos derivados, calculados na consulta, e não colunas.
Forma normal analisada: 3FN

Registro 4 — ESTOQUE (quantidade guardada)
No recorte da aula 6, a quantidade em estoque ficava em PRODUTO; ela já tinha sido levada para uma tabela ESTOQUE por lote. Mas ESTOQUE ainda guardava a quantidade como um número alterado a cada venda e entrada. Esse número é resultado das entradas e saídas, ou seja, depende de outros registros, e não só da chave do lote: guardá-lo cria redundância e risco de o saldo não bater com o que realmente aconteceu. Correção (concluída após a devolutiva): ESTOQUE virou LOTE, e cada entrada, reserva, venda, devolução, perda e ajuste passou a ser um registro em MOVIMENTACAO_ESTOQUE; o saldo passou a ser derivado.
Forma normal analisada: 3FN (atributo que não depende só da chave) e redundância

6. Antes e depois da normalização

Estrutura original
ESTOQUE(id_estoque, lote, data_entrada, data_validade, quantidade, id_produto)

Estrutura corrigida

LOTE(id_lote, codigo_lote, data_entrada, data_validade, id_produto)
MOVIMENTACAO_ESTOQUE(id_movimentacao, tipo, quantidade, data_hora, motivo, id_lote, id_funcionario, id_pedido + id_produto, id_compra + id_produto)

Justificativa técnica

Na estrutura original, a quantidade era sobrescrita a cada operação. Se duas vendas simultâneas alterassem o mesmo registro, uma podia apagar a outra, e não havia como saber por que o saldo mudou, de qual lote saiu cada venda ou quanto estava reservado para a loja virtual. Com a decomposição, os dados fixos do lote (código, validade, entrada) ficam em LOTE, e cada quantidade movimentada vira um fato próprio em MOVIMENTACAO_ESTOQUE, que não é alterado nem excluído (RN19). O saldo é calculado a partir dessas movimentações, então não existe mais um número que possa divergir do histórico. Os relacionamentos são preservados: PRODUTO 1:N LOTE e LOTE 1:N MOVIMENTACAO_ESTOQUE, e cada venda continua ligada ao lote de onde saiu (RN20).

7. Atualização da documentação do ERP

Checklist de entrega

☑ Modelo lógico atualizado.
☑ Dicionário de dados atualizado.
☑ PK e FK conferidas.
☑ Relacionamentos revisados.
☑ Correções de normalização justificadas.
☑ Coerência entre modelo lógico e dicionário de dados verificada.

8. Validação com o professor

Correção a apresentar: ESTOQUE com a quantidade guardada, decomposto em LOTE e MOVIMENTACAO_ESTOQUE (seção 6).

Observações da validação

Visto do professor (no documento impresso): _____

9. Participação e permanência

Contribuições dos integrantes

Ausências e não participação na segunda parte da aula

10. Reflexão final — Aprendizagem significativa

Por que a criação de tabelas não garante, isoladamente, um banco de dados corretamente projetado?

Porque dividir os dados em tabelas não garante que cada informação esteja no lugar certo. Se uma tabela guarda atributos que dependem de outra coisa que não a sua chave (como a validade em PRODUTO, que depende do lote), surgem redundância e anomalias: o mesmo dado repetido em várias linhas, que pode ser atualizado em uma e esquecido em outra. A normalização usa as dependências funcionais para verificar se cada atributo depende da chave, da chave inteira e só da chave (1FN, 2FN e 3FN). Isso, junto com as PKs, FKs e restrições, é o que mantém a integridade dos dados. No nosso caso, a tabela ESTOQUE “existia” e funcionava, mas só a análise de dependências mostrou que a quantidade não podia ser um número guardado e precisava vir das movimentações de cada lote.

Visto do professor: _____ Horário: _____